

TECHNICAL TRAINING COURSE

JAXB —

Java API for XML Binding

- Data Binding · XML Schema · Two Paths
- JAXB Compilation · Type Mappings
- Annotations · Marshal/Unmarshal · Object Models

Java 11 LTS

Tomcat 9.0

JAXB 2.3

Jakarta EE



COURSE AGENDA

8 Sections — 1 Day — Hands-On Labs

01 The Need for Data Binding

02 XML Schema

03 Two Paths

04 JAXB Compilation

05 Mapping Schema Types to Java

06 Java to XML Mapping (Annotations)

07 Marshalling and Unmarshalling

08 Working with JAXB Object Models

The Need for Data Binding

Why manual XML parsing doesn't scale

Without JAXB — Manual DOM

- Create DocumentBuilder + empty Document
- Create every element and attribute manually
- Append nodes to correct parents
- Write with Transformer to OutputStream
- Repeat ALL of that in reverse for parsing
- Update every piece on schema change

With JAXB — One Method Call

- `marshaller.marshal(order, outputStream)`
- `unmarshaller.unmarshal(inputStream)`
- Type-safe Java objects
- Built-in schema validation
- Regenerate on schema change
- IDE support + debuggable objects

JAXB Versions & Java 11 Dependency

Version	Spec	Included In	Java 11 Status
JAXB 1.0	JSR 31	J2EE 1.4	Obsolete
JAXB 2.0	JSR 222	Java EE 5 / JDK 6	Removed in JDK 9
JAXB 2.2	JSR 222	Java EE 7 / JDK 8	Removed in JDK 9
JAXB 2.3	JSR 222	Jakarta EE / Maven	✓ Add as Maven dep
JAXB 3.x	Jakarta	Jakarta EE 9+	Uses javax.xml.bind.* namespace

Java 11 / Tomcat 9 — Required Maven Dependency:

```
javax.xml.bind:jaxb-api:2.3.1 + com.sun.xml.bind:jaxb-impl:2.3.9 (runtime scope)
```

XML Schema — Key Constructs

xs:simpleType

Named type derived from a built-in; adds restrictions

<xs:restriction base='xs:string'> + pattern/enumeration/length

xs:complexType

Type with child elements and/or attributes

xs:sequence / xs:all / xs:choice with xs:element children

xs:element

Declares an element — may reference a type or define inline

minOccurs / maxOccurs control cardinality

xs:attribute

Declares an attribute on a complexType

use='required' | 'optional' | 'prohibited'

xs:extension

Extends an existing complexType — maps to Java inheritance

<xs:complexContent><xs:extension base='tns:BaseType'>

xs:import

Imports types from a different namespace

namespace + schemaLocation attributes required

XSD Built-In Type → Java Mapping

XSD Type	Java Type	Note
<code>xs:string</code>	<code>String</code>	Always reference; null for absent optional
<code>xs:int</code>	<code>int</code> / <code>Integer</code>	Unboxed if required; boxed if optional
<code>xs:long</code>	<code>long</code> / <code>Long</code>	Same boxing rule as <code>xs:int</code>
<code>xs:decimal</code>	<code>BigDecimal</code>	Exact arithmetic — use for all monetary values
<code>xs:double</code>	<code>double</code> / <code>Double</code>	IEEE 754 — avoid for money
<code>xs:boolean</code>	<code>boolean</code> / <code>Boolean</code>	
<code>xs:date</code> / <code>dateTime</code>	<code>XMLGregorianCalendar</code>	Use <code>XmlAdapter</code> to map to <code>java.time.LocalDate</code>
<code>xs:base64Binary</code>	<code>byte[]</code>	JAXB auto-encodes/decodes Base64
<code>xs:positiveInteger</code>	<code>BigInteger</code>	Arbitrary precision integer > 0

Key: `xs:decimal` → `BigDecimal` is critical for financial data. `xs:date` → use `XmlAdapter` to map to `java.time.LocalDate` in Java 11.

Two Development Paths

Schema-First vs Java-First — choose before you write a line of code

Path 1 — Schema-First (XSD → Java)

Start: XSD document(s)

Tool: xjc compiler

Generates: annotated Java POJOs

Contract is authoritative — highest interop

Best for: B2B, SOAP, industry standards

Risk: schema changes trigger regeneration

Path 2 — Java-First (Annotations → XML)

Start: Annotated Java classes

Tool: JAXB annotations + schemagen

Generates: XSD from Java (optional)

Fastest path to get started

Best for: internal tools, rapid iteration

Risk: annotation drift from contract

JAXB Compilation — xjc & Generated Artifacts

xjc command (Java 11 — standalone distribution required):

```
xjc -p com.example.model -d src/main/java src/main/xsd/orders.xsd
xjc -p com.example.model -b bindings.xjb -d src/main/java orders.xsd
```

Generated artifacts:

Order.java

Annotated POJO for OrderType complexType

ObjectFactory.java

create*() factory methods for every type

StatusType.java

Java enum from XSD enumeration simpleType

package-info.java

@XmlSchema — namespace & elementFormDefault

Never hand-edit generated classes — they are overwritten on the next xjc run. Use binding customisation files (.xjb) instead.

Binding Customisation Files (.xjb)

```
<!-- bindings.xjb - customise xjc output without editing generated code -->
<jaxb:bindings xmlns:jaxb="http://java.sun.com/xml/ns/jaxb"
               xmlns:xs="http://www.w3.org/2001/XMLSchema"
               version="2.1" schemaLocation="orders.xsd">

  <!-- Rename a generated class -->
  <jaxb:bindings node="//xs:complexType[@name='OrderType']">
    <jaxb:class name="PurchaseOrder"/>
  </jaxb:bindings>

  <!-- Rename a generated property -->
  <jaxb:bindings node="//xs:element[@name='lineItem']">
    <jaxb:property name="lineItems"/>
  </jaxb:bindings>

  <!-- Map xs:date to java.time.LocalDate via XmlAdapter -->
  <jaxb:bindings node="//xs:element[@name='orderDate']">
    <jaxb:property>
      <jaxb:baseType name="java.time.LocalDate"/>
    </jaxb:property>
  </jaxb:bindings>

</jaxb:bindings>
```

Java to XML — Annotation Reference

JAXB 2.3 annotations | package: jakarta.xml.bind.annotation

`@XmlRootElement`

Top-level element — name, namespace

`@XmlType`

ComplexType mapping — propOrder, name, namespace

`@XmlAccessorType`

Access strategy: FIELD|PROPERTY|PUBLIC_MEMBER|NONE

`@XmlElement`

Child element — name, required, nillable, defaultValue

`@XmlAttribute`

XML attribute — name, required, namespace

`@XmlValue`

Text content of element (simpleContent)

`@XmlTransient`

Exclude field from all XML mapping

`@XmlElementWrapper`

Wraps collection: <items><item/></items>

`@XmlJavaTypeAdapter`

Custom XmlAdapter for type conversion

`@XmlEnum` / `@XmlEnumValue`

Enum → XSD enumeration mapping

Annotating a Domain Class — Java-First Path

```
@XmlRootElement(name = "product")
@XmlType(propOrder = {"sku", "name", "price", "category"})
@XmlAccessorType(XmlAccessType.FIELD)
public class Product {

    @XmlAttribute(name = "sku", required = true)
    private String sku;

    @XmlElement(required = true)
    private String name;

    @XmlElement(required = true)
    @XmlJavaTypeAdapter(MoneyAdapter.class)
    private BigDecimal price;

    @XmlElement(name = "cat")
    private String category;

    @XmlTransient
    private String internalCode; // excluded

    @XmlElementWrapper(name = "tags")
    @XmlElement(name = "tag")
    private List<String> tags;
}
```

Produces:

```
<product sku="W-101">
  <name>Blue Widget</name>
  <price>9.99</price>
  <cat>Widgets</cat>
  <tags>
    <tag>sale</tag>
    <tag>new</tag>
  </tags>
</product>
```

Marshalling — Java → XML

JAXBContext is thread-safe — create once. Marshaller is NOT — create per operation.

```
// Create once (static field or singleton bean)
JAXBContext ctx = JAXBContext.newInstance(Order.class);

// Per-operation — NOT thread-safe
Marshaller m = ctx.createMarshaller();
m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true); // pretty-print
m.setProperty(Marshaller.JAXB_ENCODING, "UTF-8");

// To String
StringWriter sw = new StringWriter();
m.marshal(order, sw);
String xml = sw.toString();

// To File
m.marshal(order, Path.of("order.xml").toFile());

// To OutputStream (HTTP response — Java 11 / Tomcat 9)
resp.setContentType("application/xml;charset=UTF-8");
m.marshal(order, resp.getOutputStream());
```

Unmarshalling + Schema Validation

```
// Unmarshal from InputStream (Tomcat 9 request body)
Unmarshaller u = ctx.createUnmarshaller();

// Attach XSD schema for validation
SchemaFactory sf = SchemaFactory.newInstance(XMLConstants.W3C_XML_SCHEMA_NS_URI);
Schema schema = sf.newSchema(getClass().getResource("/xsd/orders.xsd"));
u.setSchema(schema);

// Custom event handler — fail fast on ERROR or FATAL
u.setEventHandler(event -> {
    if (event.getSeverity() >= ValidationEvent.ERROR) {
        throw new RuntimeException(
            "Validation failed at line " +
            event.getLocator().getLineNumber() + ": " + event.getMessage());
    }
    return true; // continue on WARNING
});

// Unmarshal — throws UnmarshalException on invalid XML
try (InputStream is = request.getInputStream()) {
    Order order = (Order) u.unmarshal(is);
    processOrder(order);
} catch (JAXBException e) {
    response.sendError(400, e.getMessage());
}
```

JAXB in a Tomcat 9 Servlet

Java 11 — initialise JAXBContext in a @WebListener, not inside doGet/doPost

@WebListener
contextInitialized()

JAXBContext
(singleton in SC)

@WebServlet
doPost()

createUnmarshaller()
unmarshal(is)

```
@WebListener
public class JaxbContextListener implements ServletContextListener {
    public void contextInitialized(ServletContextEvent sce) {
        try {
            JAXBContext ctx = JAXBContext.newInstance(Order.class);
            sce.getServletContext().setAttribute("jaxbContext", ctx);
        } catch (JAXBException e) { throw new RuntimeException(e); }
    }
}
```

StAX + JAXB — Streaming Large Documents

Use StAX cursor + `unmarshal(reader, Class)` for large batch files — avoids loading the full document into memory.

```
// Disable XXE — required security setting (Java 11)
XMLInputFactory xif = XMLInputFactory.newInstance();
xif.setProperty(IS_SUPPORTING_EXTERNAL_ENTITIES, false);
xif.setProperty(SUPPORT_DTD, false);

Unmarshaller u = ctx.createUnmarshaller();

try (InputStream is = Files.newInputStream(path)) {
    XMLStreamReader xsr = xif.createXMLStreamReader(is);
    while (xsr.hasNext()) {
        xsr.next();
        if (xsr.isStartElement()
            && "order".equals(xsr.getLocalName())) {
            // Unmarshal just this <order> element
            JAXBElement<Order> je =
                u.unmarshal(xsr, Order.class);
            processOrder(je.getValue());
        }
    }
}
```

Use StAX+JAXB when:

- File > 1 MB
- 10,000+ root elements
- Memory is constrained
- Streaming pipeline

Use full `unmarshal()` when:

- Small documents
- Single root element
- Simplicity matters

Performance & Best Practices



Singleton JAXBContext

Create once; store as static final or app-scoped bean. Creation is expensive.



Per-request Marshaller

Not thread-safe — create a new Marshaller/Unmarshaller for each operation.



Cache compiled Schema

Compile Schema once from SchemaFactory; reuse across requests.



Disable FORMATTED_OUTPUT in prod

Pretty-printing adds significant whitespace overhead; use false in production.



Stream to OutputStream

Avoid intermediate String; marshal directly to response OutputStream.



@XmlTransient aggressively

Exclude internal/sensitive fields to reduce payload size and prevent data leakage.



StAX for large docs

Use StAX cursor + unmarshal(reader, Class) for files > 1 MB.



Disable XXE always

Set IS_SUPPORTING_EXTERNAL_ENTITIES=false on XMLInputFactory in all XML parsing code.

Key Takeaways

01

JAXB must be added as a Maven dependency on Java 11 — it was removed from the JDK in Java 9.

02

Schema-first (xjc) is best for B2B and partner integrations; Java-first suits internal tools.

03

xjc generates Order, ObjectFactory, enums, and package-info.java from a single XSD file.

04

xs:decimal → BigDecimal is critical; xs:date → use XmlAdapter to map to java.time.LocalDate.

05

@XmlAccessorType(FIELD) + explicit annotations gives the most predictable XML mapping.

06

JAXBContext is thread-safe — create once. Marshaller/Unmarshaller are NOT — create per call.

Thank You

Questions & Lab Time

Lab exercises:

- Lab 1: xjc — compile the orders schema
- Lab 2: Annotate a domain class (Java-first)
- Lab 3: Marshal and unmarshal with validation
- Lab 4: StAX+JAXB batch processing exercise
- Pass mark: 70% for certification

